Team 15
CSE 361
Dr. Karim Emara



Ainshams University

Multiplayer Game State Synchronization

Mini RFC

June 15, 2025

Team Members

Tarek Mohamed Atef 2300245
Nour Mamdouh Mahmoud 24P0184
Mai Hamed Hussien 23P0261
Mariam Hamdy Hassan 23P0260
Rana Tarek Ahmed 23P0264
Mariam Riyad Freeg 23P0147

1. Introduction

The **Grid State Synchronization Protocol (GSSP)** is a lightweight, event-driven protocol for real-time multiplayer games such as *Grid Clash*, where players compete to claim cells on a shared grid. Unlike traditional TCP or continuous streaming approaches, GSSP operates over **UDP** and transmits data only when player actions occur, reducing bandwidth and ensuring low-latency updates.
GSSP follows a **client–server model**, with an authoritative server maintaining the global grid state and broadcasting snapshots when board state changes occur. Clients send discrete action requests (e.g., cell claims) to the server and apply received snapshots to stay synchronized.

Assumptions and Constraints

- **Network Model:** Unreliable datagram delivery using UDP/IP.
- **Packet Loss:** Occasional loss is acceptable; reliability is managed through sequence numbers and acknowledgment numbers.
- **Ordering:** Out-of-order packets may occur and are resolved using sequence numbers.
- **Maximum Packet Size:** Messages are designed to remain below **512 bytes**, ensuring safe transmission within typical MTU limits and avoiding fragmentation.
- **Max Player Count: The protocol is designed for cells each with 8 states 1 state for empty and 7 states for players.**

In summary, GSSP provides an efficient, low-overhead synchronization layer that balances responsiveness and bandwidth economy for event-driven multiplayer environments.

2. Protocol Architecture

2.1 Overview

The **Grid State Synchronization Protocol (GSSP)** is a lightweight, event-driven protocol built on top of **UDP/IP**.
It enables multiple clients to interact in a shared, grid-based environment where each cell can be claimed or modified by player actions.
Because the game state changes only when a player performs an action (e.g., clicking a cell), GSSP avoids periodic input streaming and instead uses discrete event messages.
This minimizes bandwidth usage while maintaining low-latency synchronization between all participants.

2.2 Entities

- **Game Server (Authoritative Node):**
  Maintains the authoritative game state, processes incoming player actions, and broadcasts board state to all connected clients when it changes only.
- **Game Client:**
  Sends local player inputs (claims) to the server and applies received state updates (from the server) to render other players' claims and game events.

2.3 Communication Model

GSSP follows a **client–server model** over UDP:

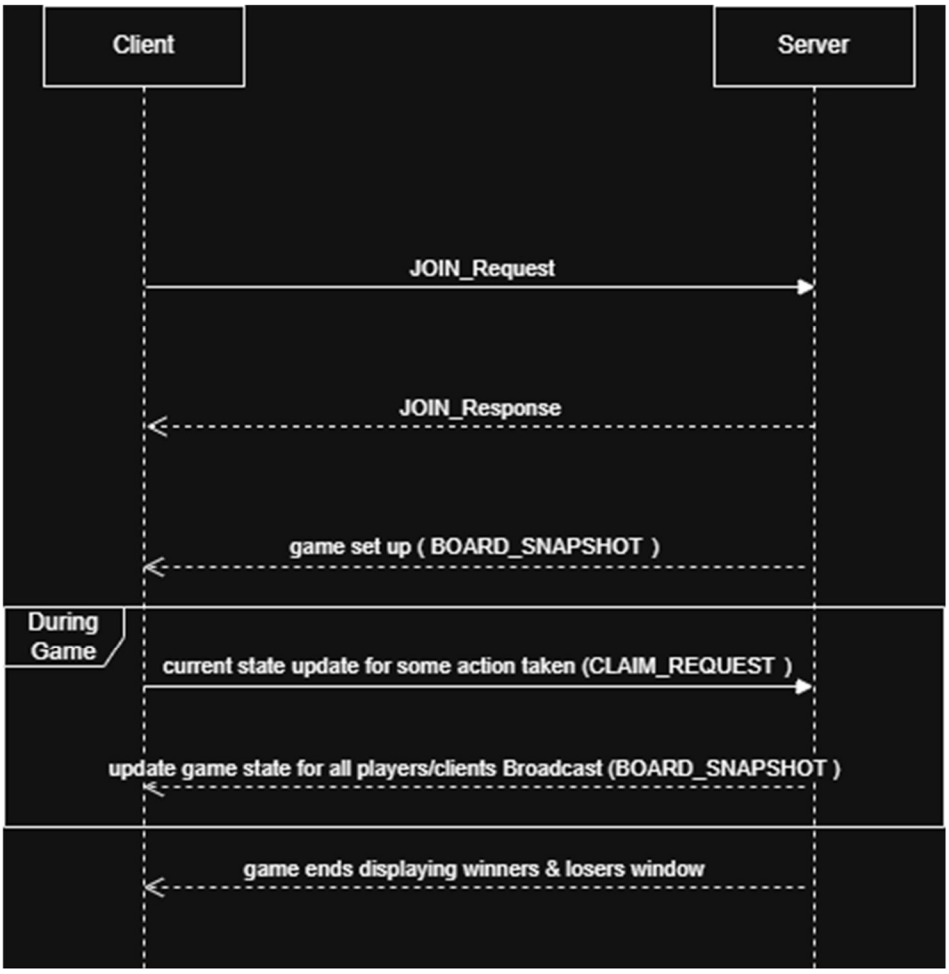
1. **Client -> Server:**
   - Sends an **Action Request** (e.g., CLAIM_REQUEST) only when the player initiates an action.
   - Includes target cell coordinates, and timestamp. Timestamps are useful when two packets arrive, but the one with earlier timestamp if arrives late has no effect.
2. Server -> Clients
   - Broadcast **Board State** messages to all clients only when board state changes.

2.4 Layering

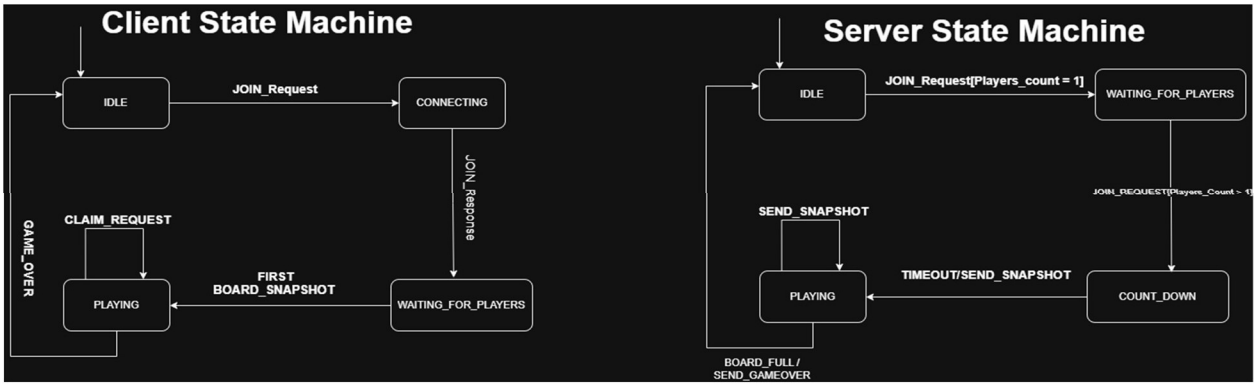
GSSP is implemented at the **application layer**, built directly on top of **UDP/IP**.
It does not provide built-in reliability or congestion control; instead, it uses timers, acknowledgments and sequence numbers to detect loss and manage packet ordering respectively.

Layer	Protocol	Purpose
Application	**GSSP**	Game state sync, player actions
Transport	**UDP**	Connectionless, low-latency delivery
Network	**IP**	Routing between hosts

2.5 Sequence Diagram



2.6 State Diagram



3. Message Formats

3.1 Header Table

All GCSSP messages are transmitted over **UDP** datagrams.
Each message begins with a fixed-size **header = 28 bytes** followed by a **payload** whose contents depend on the message type.

Field	Size (bytes)	Byte Offset	Format Code	Description
Protocol_id	4	**0**	**4s**	4-byte ASCII unique identifier ('GSSP')
Version	1	**4**	**B**	Protocol version number (current = 1)
MsgType	1	**5**	**B**	Type of message (e.g., 0 = JOIN_REQUEST, 1 = JOIN_RESPONSE, 2 = CLAIM_REQUEST, 3 = BOARD_SNAPSHOT, 4 = GAME_OVER, 5 = LEAVE, 6: GAME_START, 7 = WAITING_ROOM, 8 = ACK, 9 = LEADERBOARD)
payload_len	2	**6**	**H**	Total payload length in bytes
SnapshotID	2	**8**	**H**	Unique identifier for the snapshot
SeqNum	4	**10**	**I**	Sequence number for reordering
AckNum	4	**14**	**I**	Acknowledgment for detecting loss
Timestamp	8	**18**	**Q**	Milliseconds since Unix epoch
Checksum	2	**26**	**H**	16-bit Internet Checksum

Variable-Length Payloads & MTU Limit: GSSP supports variable-length payloads, with the size explicitly defined in the payload_len header field.

Fragmentation Rule: To avoid IP fragmentation, the protocol enforces a strict total packet size limit of **512 bytes** (Header + Payload).

- **Header Size:** 28 bytes.
- **Max Safe Payload:** 484 bytes.

4. Communication Procedures

This section describes how Grid Clash endpoints initiate communication, exchange game data, recover from loss, and terminate sessions. The protocol uses UDP with **Selective Repeat (SR)** reliability on top of application-level sequence numbers and acknowledgments. All messages follow the header format defined in Section 3.

4.1 Session Start

Because UDP is connectionless, Grid Clash uses a lightweight handshake to establish a logical session and assign a player_id.

1. **Client → Server: JOIN_REQUEST (MsgType = 0)**
   SeqNum = C0, AckNum = 0.
2. **Server → Client: JOIN_RESPONSE (MsgType = 1)**
   SeqNum = S0, AckNum = C0. Payload includes assigned player ID and the client enters the **Waiting Room** state.
   - If the game is active and not full, the server immediately sends **GAME_START** (MsgType = 6).
   - If the game is waiting for players, the server broadcasts **GAME_START** only when the minimum player count (2) is reached.
3. **Client** sends next message with **AckNum = S0**, confirming receipt and completing handshake.

4.2 Normal Data Exchange

Game actions and state snapshots flow using Selective Repeat ARQ. Conflict resolution is handled via timestamps to ensure "**Last-Write-Wins**" semantics.

- **SeqNum** identifies each message uniquely.
- **AckNum** acknowledges *all correctly received messages*, possibly non-contiguous.
- A **receive window** stores out-of-order packets.
- A **send window** keeps unacknowledged packets for retransmission.

1. **Client → Server (CLAIM_REQUEST, MsgType = 2)**
   Sent whenever the player selects a cell. *SeqNum* increases per request; *AckNum* acknowledges the latest valid server snapshot.
   - Conflict Resolution: The server validates the Timestamp field. If the timestamp is older than the current cell's last modification time, the request is discarded to prevent race conditions.
   - **Optimistic Prediction:** To ensure a responsive user experience, the client applies the move locally immediately after sending the CLAIM_REQUEST. If the request fails (e.g., due to packet loss or server rejection), the client reverts the cell to its previous state upon receiving the next authoritative BOARD_SNAPSHOT.
2. **Server → Client (BOARD_SNAPSHOT, MsgType = 3)**
   Sent after processing claims, contains whole board.
   SnapshotID increments with each new board state.

4.3 Error Recovery (Selective Repeat)

Loss Detection

- Receivers continue to acknowledge *all packets received*, even if out of order.
- All timeouts are treated as lost packets

Retransmission Rule

A sender retransmits a message if:

- Its SeqNum is unacknowledged after a timeout.

Out-of-Order Handling

- Receiver stores packets in a reorder buffer.
- Once the missing sequence number arrives, buffered packets are delivered in order.

4.4 Shutdown

Client-Initiated Shutdown

1. Client → Server: LEAVE (MsgType = 5) Includes SeqNum and AckNum.
2. Server → Client: ACK (MsgType = 8)

Server-Initiated Shutdown

1. Server broadcasts **GAME_OVER** (MsgType = 4) Payload is empty. Signals the end of the match (e.g., time expired).
2. Server broadcasts **LEADERBOARD** (MsgType = 9) Payload contains final scores and player ranks.

5.Reliability Performance Features

This protocol implements a **Selective Repeat ARQ** reliability layer optimized for real-time multiplayer state synchronization. Reliability focuses on recovering lost packets without blocking the delivery of later updates.

5.1 Sliding Window

The sender maintains a **fixed-size sliding window** of outstanding unacknowledged packets.
Let:

- **base** = lowest unacknowledged sequence number
- **nextSeqNum** = next sequence number available to send
- **N =** window size (size = 6)

Rules:

1. The sender may send packets while
   nextSeqNum < base + N.
2. All packets in the window are sent once when they first enter the window.
3. All packets in the window are sent once when they first enter the window.
4. The window advances **only when the base packet is acknowledged**.
5. **Stuck Window Recovery:** To prevent the game state from freezing due to a persistently lost

ACK, the sender monitors the oldest unacknowledged packet. If it remains unacknowledged for **3 × RTO**, the sender forcibly slides the window forward (base = base + 1) and discards the stuck packet.

This design avoids bursty retransmission and simplifies ordering.

5.2 Independent Per-Packet Timers

Unlike Go-Back-N, each packet in the SR window maintains **its own retransmission timer**:

- When a packet is first sent, a timer starts for that specific sequence number.
- If the packet is acknowledged, its timer is canceled.
- Timers of other packets continue running independently.

This allows retransmissions to be targeted and avoids resending burst ranges of data.

5.3 Retransmission Strategy

Retransmission behavior is strictly:

1. **Sender transmits all packets that enter the window once.**
2. **Each unAcked packet is monitored by its own timer.**
3. When **any packet's timer expires**, the sender:
    - Retransmits **only that expired packet**,
    - Resets its timer,
    - Keeps the rest of the window unchanged.
4. When the **base** packet is ACKed:
    - Slide the window forward by one,
    - Start the timer for the new packets entering the window (if any).

This preserves Selective Repeat semantics while keeping the implementation simple.

5.4 Timeout (RTO) Calculation

Each packet timer uses the same RTO estimate, where *alpha=0.125* and *beta=0.25*

$$EstimatedRTT = \alpha * EstimatedRTT + (1 - \alpha) * SampleRTT$$

$$DevRTT = \beta * DevRTT + (1 - \beta) * |SampleRTT - EstimatedRTT|$$
$$RTO = EstimatedRTT + 4 * DevRTT$$

5.5 Performance Notes

- Independent per-packet timers enable precise loss recovery.
- Window-shift-only-on-base-ACK maintains ordered snapshot progression, important for consistent game state.
- Small windows prevent excessive buffering or packet storms.
- The protocol remains bandwidth-efficient: redundant retransmissions are avoided entirely.

6. Experimental Evaluation Plan

To evaluate the robustness and performance of the GSSP protocol, we have developed an automated testing

framework using Python and Bash scripts. The evaluation focuses on protocol stability under adverse network conditions simulated via the Linux kernel's Traffic Control (tc) subsystem.

6.1 Test Environment & Tools (See Appendix 10.1 for the exact netem commands used.)

- **Hardware/OS**: Tests are executed on a Linux environment (required for tc and netem).
- **Network Emulation**: We use tc qdisc (Queueing Discipline) with netem (Network Emulator) to inject artificial packet loss, delay and jitter on the loopback interface (lo).
- **Packet Capture**: tcpdump is triggered automatically for every test run to generate .pcap files for deep-packet analysis.
- **Traffic Generation**: A custom HeadlessClient simulates player behavior by sending CLAIM_REQUEST messages at a fixed rate (default: 20 Hz) **without rendering a GUI**, allowing for scalable concurrency.

6.2 Test Scenarios

We define four distinct scenarios to measure performance against a baseline. Each scenario runs for 30 seconds with 4 concurrent clients.

- **Baseline:** Ideal localhost conditions to establish maximum throughput and minimum latency.
- **Loss 2%:** Tests Selective Repeat (SR) recovery efficiency under minor loss.
- **Loss 5%:** Stress tests retransmission timers and window logic under moderate loss.
- High Latency (tc ... delay 100ms & 10ms distribution normal): Evaluates jitter handling and snapshot interpolation limits with 100ms delay and ±10ms jitter.

6.3 Metrics

Data is logged per-client in CSV format where the key performance indicators include:

- **Latency (ms):** End-to-end time from server snapshot generation to client reception.
- **Jitter (ms)**: The variance in latency between consecutive packets.
- **Retransmission Rate**: The number of retransmitted packets vs. total sent.
- **Snapshot Delivery**: Total number of successful board state updates received.

6.4 Automation Workflow

The execution is managed by run_all_tests.sh, which performs the following steps:

1. **Scenario Loop:** For each of the four scenarios (Baseline, Loss 2%, Loss 5%, High Latency):
    - **Clean:** Resets existing tc rules and kills stale processes.
    - **Impair:** Applies the specific netem rule (e.g., loss 5%) to the network interface.
    - **Capture:** Starts tcpdump to record traffic to a scenario-specific .pcap file.
    - **Execute:** Launches the Game Server and 4 Headless Clients for 30 seconds.
2. **Post-Processing:** Once **all** scenarios are completed, the script executes postprocess.py. This script parses the cumulative CSV logs from all client sessions, calculates aggregate statistics (e.g., global mean latency, error rates), and generates a summary.csv file.
3. **Visualization:** Finally, generate_plots.py is called to produce comparative bar charts and time-series graphs from the summarized data.

7. Example Use Case Walkthrough

This section traces a typical session lifecycle for "Player 1":

- **Initialization:** The client initiates connection via JOIN_REQ. The server replies with JOIN_RESP (assigning Player ID 1), which the client acknowledges. Once the lobby minimum is reached, the server broadcasts GAME_START followed immediately by the first BOARD_SNAPSHOT (ID 0) to

initialize the grid.

- **Gameplay:** The player clicks cell (5,5). The client immediately updates the local grid (Optimistic Prediction) and sends a CLAIM_REQUEST. The server validates the request timestamp, replies with an ACK (Seq 3), and broadcasts BOARD_SNAPSHOT (ID 1) confirming the new cell owner to all participants.
- **Termination:** Upon timer expiry, the server broadcasts GAME_OVER. The client stops gameplay and initiates a 2-second fallback timer. Shortly after, the server sends the LEADERBOARD message with final ranks (e.g., P1 score: 4..etc), which the client displays to conclude the session.

8. Limitations & Future Work

While GSSP effectively handles real-time synchronization for the Grid Clash environment, the current implementation involves trade-offs regarding scalability, and network adaptability.

Known Limitations

- **Fixed Congestion Control:** The Selective Repeat window size is fixed at N=6. While this simplifies implementation, it does not adapt to varying network conditions. In a high-bandwidth, low-latency environment, the throughput is artificially limited. Conversely, in a highly congested network, a fixed window may contribute to packet storms rather than throttling back.
- **Player Count Constraint:** The current game logic enforces a strict limit of 4 or less active players per session. While the underlying 4-bit data structure could theoretically support up to 15 unique IDs, the server's lobby management and UI rendering are currently optimized for a maximum of 4 simultaneous participants to ensure gameplay stability. Scaling beyond this would require updates to the server's join logic and the client's rendering loop.

Future Work

- **Delta Compression:** To reduce bandwidth consumption, the protocol could implement Delta Encoding. Instead of sending the full grid, the server would send a packet containing only the coordinates and new values of cells that have changed since the last acknowledged snapshot (e.g., SNAPSHOT_DELTA message type).
- **Dynamic Window Sizing**: Implementing a dynamic congestion control algorithm (similar to TCP's AIMD—Additive Increase, Multiplicative Decrease) would allow the protocol to expand the window size when the network is healthy and shrink it when packet loss is detected, maximizing throughput without causing congestion.

9. References

1. **[RFC 908]** Velten, D., Hinden, R., and J. Sax, "Reliable Data Protocol", RFC 908, July 1984.
- *Used for: General concepts of sequence numbers and connection management in reliable packet transmission.*
2. **[RFC 6298]** Paxson, V., Allman, M., Chu, J., and M. Sargent, "Computing TCP's Retransmission Timer", RFC 6298, June 2011.
- *Used for: The EWMA formulas (alpha=0.125, beta=0.25) and Karn's Algorithm logic used to calculate the Retransmission Timeout (RTO).*
3. **[RFC 908]** Velten, D., Hinden, R., and J. Sax, "Reliable Data Protocol", RFC 908, July 1984.

Used for: General concepts of sequence numbers and connection management in reliable packet transmission.

10. Appendix

10.1 Experimental Setup & Reproducibility

To ensure the experimental results in Section 6 are reproducible, the project includes a fully automated testing suite (run_all_tests.sh) that manages the network environment and test execution.

- **Network Emulation:** We utilized the Linux Traffic Control (**tc**) subsystem with **netem** to simulate

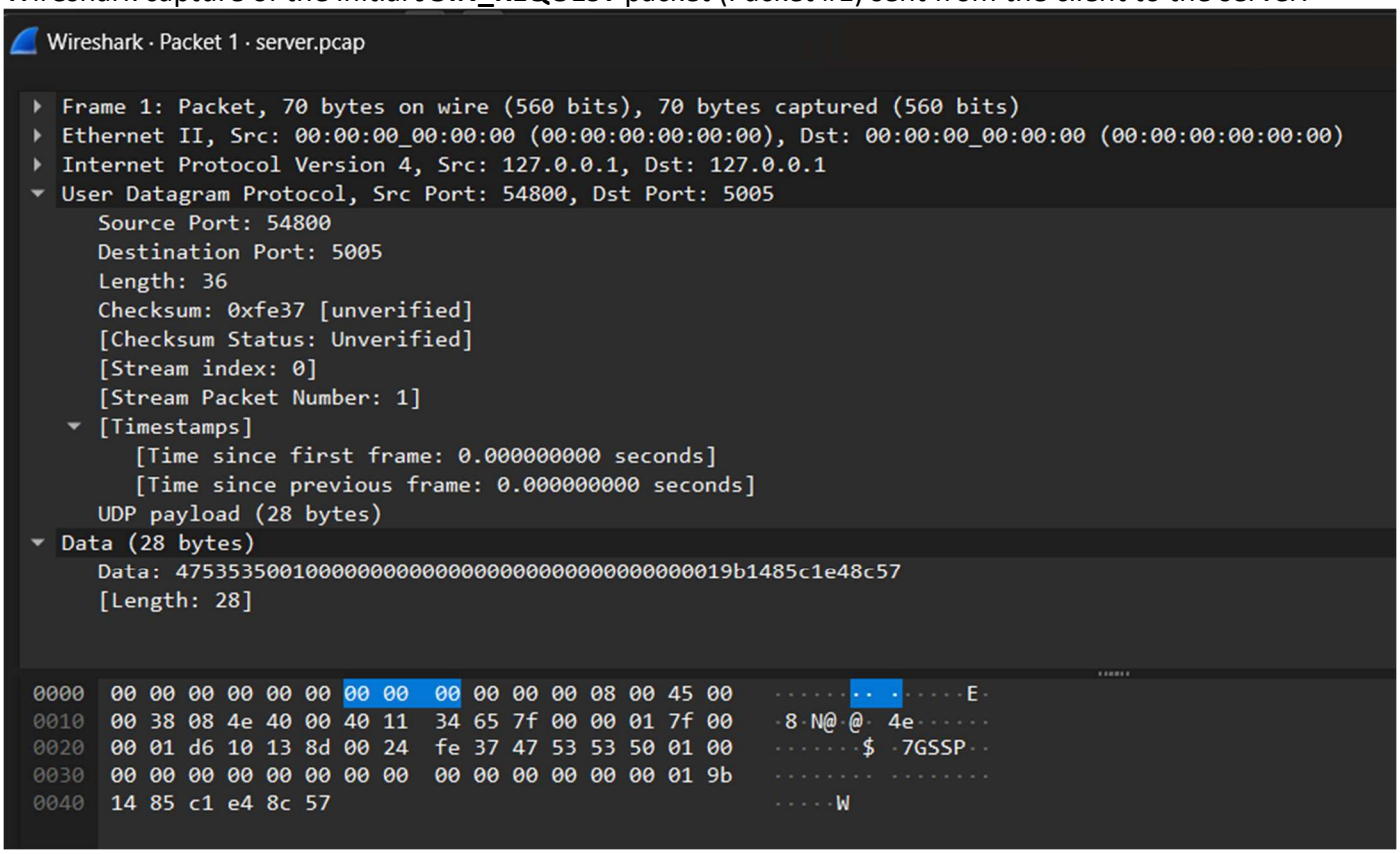
adverse conditions. The exact commands injected by our automation script are:

- **Packet Loss: sudo tc qdisc** add dev lo root netem loss <rate>% (used for 2% and 5% scenarios).
- **High Latency: sudo tc qdisc** add dev lo root netem delay 100ms 10ms distribution normal (simulating 100ms lag with jitter).
- **Baseline: sudo tc qdisc** del dev lo root (to restore localhost performance).

- **Reproducibility Checklist:**
  - **Automation:** The run_all_tests.sh script sets up the environment, launches the server and headless clients for each scenario, and cleans up processes automatically.
  - **Data Capture:** Raw metrics are logged to timestamped directories in CSV format**.**
  - **Analysis:** After all tests are complete, the postprocess.py and generate_plots.py scripts parse the aggregated logs to regenerate the exact latency and throughput plots used in the evaluation.

10.2 Packet Capture Excerpt

To validate the protocol implementation, we captured the traffic for a single session. Below is the Wireshark capture of the initial **JOIN_REQUEST** packet (Packet #1) sent from the client to the server:



Explanation of Captured Data: The highlighted packet contains the 28-byte GSSP Header with no payload, confirming the format defined in Section 3:

- **Protocol ID (0x47535350):** The first 4 bytes decode to **ASCII "GSSP".**
- **Version (0x01):** Sets the protocol version to 1.
- **MsgType (0x00):** Identifies this as a **JOIN_REQUEST.**
- **Payload Length (0x0000):** Indicates no payload follows the header.
- **Checksum & Timestamp:** The remaining bytes contain the sequence number (0), acknowledgment number (0), generation timestamp, and the 16-bit integrity checksum.